

Query Processing

Query Processing is the series of steps a database management system (DBMS) performs to execute a query and return results. It involves:

1. Parsing & Translation

- The SQL query is checked for syntax errors.
- The query is translated into an internal representation (like a **relational algebra expression**).

2. Optimization

- The DBMS finds the **most efficient way** to execute the query.
- This may include choosing indexes, join methods, or the order of operations.

3. Execution

- The DBMS executes the query plan and returns the results.

Key point: Query processing converts a high-level SQL query into a low-level execution plan using relational algebra.

2. Query Trees

A **Query Tree** is a **tree representation of a SQL query** in terms of **relational algebra operations**.

- **Leaves:** base tables
- **Intermediate nodes:** relational algebra operations like σ (select), π (project), $\bowtie$ (join)
- **Root:** final result

Query trees help the DBMS **visualize and optimize** queries.

Example in SQL Server

Suppose we have two tables:

```
CREATE TABLE Employees (
```

```
EmpID INT PRIMARY KEY,  
Name NVARCHAR(50),  
DeptID INT,  
Salary INT  
);
```

```
CREATE TABLE Departments (  
    DeptID INT PRIMARY KEY,  
    DeptName NVARCHAR(50)  
);
```

And we write a query:

```
SELECT e.Name, d.DeptName  
FROM Employees e  
JOIN Departments d ON e.DeptID = d.DeptID  
WHERE e.Salary > 50000;
```

Step 1: Convert SQL to Relational Algebra

1. **Selection:** $\sigma_{\text{Salary} > 50000}(\text{Employees})$
2. **Join:** $(\sigma_{\text{Salary} > 50000}(\text{Employees})) \bowtie \text{Departments on DeptID}$
3. **Projection:** $\pi_{\text{Name}, \text{DeptName}}(\dots)$

Step 2: Draw the Query Tree

$\pi_{\text{Name}, \text{DeptName}}$
|
 $\bowtie_{\text{DeptID}}$
/ \

σ Salary > 50000 Departments

|

Employees

Explanation:

1. Start at **leaves** (tables Employees and Departments)
2. Apply **selection** (Salary > 50000) on Employees
3. Perform **join** with Departments on DeptID
4. Apply **projection** to return only Name and DeptName

Step 3: Query Processing in SQL Server

- SQL Server internally converts your query into a **query tree** (relational algebra)
- Then, it generates a **Query Execution Plan**.

You can see it using:

-- Display the actual execution plan

SET SHOWPLAN_ALL ON;

GO

SELECT e.Name, d.DeptName

FROM Employees e

JOIN Departments d ON e.DeptID = d.DeptID

WHERE e.Salary > 50000;

GO

SET SHOWPLAN_ALL OFF;

- SQL Server will **not execute** the query but show the **execution plan**, which corresponds to the **query tree**.
-

4 Major Stages of Query Processing

1 Smart → Submission & Parsing

- Send Query
 - Network Interface (SNI receives TDS packet)
 - Parse and Algebrize (syntax & logical tree)
- 👉 Remember: "Send → Network → Parse → Algebrize"
-

2 Owls → Optimization & Plan Generation

- Optimizer analyzes logical tree
 - Plan is created
 - Cache checked for reuse
- 👉 Remember: "Optimize → Plan → Cache"
-

3 Eat → Execution

- Executor runs the plan
 - Access methods fetch data (Buffer Pool / Disk I/O)
 - Transaction Manager logs changes
- 👉 Remember: "Execute → Access → Transaction"
-

4 Rice → Result Delivery

- Result set built
 - In TDS packets
 - Client receives
- 👉 Remember: "Result → In TDS → Client"

Query Optimization:

Query Optimization is the process of choosing the most efficient way to execute a SQL query.

A query can be executed in many ways (different join orders, index usage, etc.), but the goal is to find the one with minimum cost — i.e., minimal CPU, memory, and I/O.

There are two main approaches:

1. Heuristic (Rule-based) optimization – uses predefined rules.
2. Cost-based optimization – uses statistics and cost estimation.

What is Heuristic Optimization?

Heuristic Query Optimization uses rules of thumb (heuristics) to transform a query into an equivalent but more efficient form — *without* computing the actual cost of each plan.

It's like saying:

“We know this transformation usually helps, so let's apply it!”

Common Heuristics for Query Optimization

Here are the main heuristic rules used by DBMSs (and you can apply these mentally when designing queries).

1 Perform Selections Early (Push Selections Down)

Move WHERE conditions as close as possible to the base tables.

Example:

-- Less efficient

```
SELECT Name FROM Employees e
```

```
JOIN Departments d ON e.DeptID = d.DeptID
```

```
WHERE e.Salary > 50000;
```

-- Better (selection before join)

-- Conceptually: $\sigma_{Salary > 50000}(Employees) \bowtie Departments$

Why:

It reduces the number of rows early, so subsequent joins or operations process less data.

2 Perform Projections Early (Eliminate Unused Columns)

Only select columns you actually need.

Example:

-- Less efficient

SELECT * FROM Employees;

-- Better

SELECT EmpID, Name FROM Employees;

Why:

Smaller intermediate tables = less memory, faster joins.

3 Combine Cartesian Product with Selection into a Join

Avoid unnecessary Cartesian products.

Example:

-- Inefficient (Cartesian product then filter)

FROM Employees, Departments

WHERE Employees.DeptID = Departments.DeptID

-- Better

FROM Employees JOIN Departments ON Employees.DeptID = Departments.DeptID

Why:

Avoids generating a huge intermediate result before filtering.

4 Push Projections and Selections Below Joins

When possible, apply SELECT and WHERE before joining tables.

Example:

-- Inefficient: join all, then filter

```
SELECT e.Name, d.DeptName
FROM Employees e
JOIN Departments d ON e.DeptID = d.DeptID
WHERE e.Salary > 50000;
```

-- Heuristic optimized:

```
SELECT e.Name, d.DeptName
FROM (SELECT EmpID, Name, DeptID FROM Employees WHERE Salary > 50000) e
JOIN Departments d ON e.DeptID = d.DeptID;
```

Why:

The subquery reduces rows before the join.

5 Replace Nested Subqueries with Joins (When Possible)

Example:

-- Inefficient

```
SELECT Name
FROM Employees
WHERE DeptID IN (SELECT DeptID FROM Departments WHERE DeptName =
'Finance');
```

```
-- Better  
  
SELECT e.Name  
  
FROM Employees e  
  
JOIN Departments d ON e.DeptID = d.DeptID  
  
WHERE d.DeptName = 'Finance';
```

Why:

Joins are usually faster and more optimized by the query planner.

6 Reorder Joins to Join Smaller Tables First

If you have multiple joins, start joining small or filtered tables first.

Example:

```
-- Suppose Employees has 1M rows, Departments has 5 rows  
  
-- DBMS will choose Departments first
```

Why:

Smaller intermediate results → less cost for subsequent joins.

7 Use Set Operations Instead of Joins When Appropriate

Example:

```
-- Instead of OR conditions
```

```
SELECT * FROM Employees WHERE DeptID = 1 OR DeptID = 2;
```

```
-- Better (set operation)
```

```
SELECT * FROM Employees WHERE DeptID IN (1,2);
```

Why:

Set operations can be optimized internally with hashing.